Smart Snake Robot — Complete Electrical System

SURGE-090 · August 2026

Dual 3S LiPo · 10× Waveshare ST3215 Servos · ESP32 Onboard · ESP-NOW to Raspberry Pi 5 Base Station

Executive Summary

This document consolidates all electrical architecture, component specifications, wiring details, safety protocols, and system integration for the Smart Snake Robot project. The system has been comprehensively redesigned from the original Dynamixel XL330 + Raspberry Pi 5 onboard architecture to a lightweight, modular dual-controller approach:

- **Onboard Compute:** An ESP32 microcontroller mounted directly on the robot running real-time central pattern generator (CPG) gait control, sensor fusion, and high-speed servo bus communication.
- **Base Station:** A Raspberry Pi 5 operating as the high-level supervisory node, executing user interfaces, motion planning, and real-time telemetry dashboards.
- **Wireless Link:** ESP-NOW peer-to-peer 2.4 GHz link between the onboard ESP32 and a dedicated base station ESP32 radio bridge connected via USB serial to the Raspberry Pi 5.

Key Decision Drivers

- **Space & Mass Constraints:** Internal volume and payload capacity on the segmented snake robot chassis preclude mounting the Raspberry Pi 5 onboard.
- **High Torque & Availability:** Waveshare ST3215 serial bus servos deliver over 6× the torque of the original XL330 motors at approximately 1/3 the cost and are readily sourced in India.
- **Zero-Infrastructure Wireless:** ESP-NOW delivers ultra-low-latency (< 10 ms) deterministic communication without requiring external Wi-Fi routers or access points.

Contents

1 Hardware Inventory — Keep / Return 3
1.1 Keep These (Final System) 3
1.2 Return to Supplier (Dynamixel-Era Components) 3

2 Component Specifications 4
2.1 Waveshare ST3215 Smart Serial Servo 4
2.2 Bus Servo Adapter (A) — Waveshare SKU 25514 4
2.3 Orange 3S LiPo 1500 mAh 30C Battery 4
2.4 Mini560-5V Step-Down Synchronous Buck Converter 5

3 Power Architecture 5
3.1 Dual Parallel LiPo Configuration 5
3.2 Protection, Filtering & Safety Subsystems 5

4 Wiring Details — On the Robot **5**
4.1 12 V Main Power Distribution Bus 5
4.2 System Ground (GND) Topology 6
4.3 UART & Servo Bus Routing 6

5 Wiring Details — Base Station **6**

6 First Power-Up Sequence **6**

7 Servo Integration & ID Assignment **7**
7.1 Sequential ID Assignment Procedure 8
7.2 ID Assignment Source Code 8

8 Sensor Integration **9**
8.1 MPU6050 6-Axis IMU (Orientation Feedback) 9
8.2 HC-SR04P Ultrasonic Sensor (Obstacle Detection) 9
8.3 Unified Sensor Polling Code 9

9 ESP-NOW Wireless Telemetry & Command Link **10**
9.1 Phase 1: MAC Address Discovery 10
9.2 Phase 2: Robot Transmitter Firmware (ESP32 #1) 11
9.3 Phase 3: Base Station Receiver Firmware (ESP32 #2) 12

10 Project Status & Next Milestones **12**

1 Hardware Inventory — Keep / Return

1.1 Keep These (Final System)

Core Motors & Driver:
- 10× Waveshare ST3215 Smart Servos (6.0–12.6 V, 1 Mbps half-duplex TTL, 2.7 A stall, 30 kg·cm torque at 12 V).
- Bus Servo Adapter (A) — Waveshare SKU 25514 (9–12.6 V input, UART control, automated TTL direction switching, servo daisy-chain output).

Power Supply & Distribution:
- 2× Orange 3S LiPo 1500 mAh 30C battery packs (XT60 connectors, ∼Rs. 1,219 each).
- Mini560-5V synchronous step-down buck converter (high efficiency, up to 3 A continuous, ∼Rs. 150).
- 2× LiPo Low-Voltage Buzzer Alarms with configurable threshold set to 3.4 V/cell (∼Rs. 100 each).
- 2× 15 A Mini Blade Fuses + in-line waterproof fuse holders (∼Rs. 100).
- XT60 Loop Key (serves as the spark-safe master emergency power disconnect, ∼Rs. 150).
- 16 AWG high-strand silicone wire + XT60 parallel Y-harness pigtails (∼Rs. 300).
- 1000 µF 25 V Low-ESR electrolytic filter capacitor (∼Rs. 30).
- iMAX B6 professional balance charger/discharger (∼Rs. 1,500).

Microcontrollers & Compute:
- **ESP32 Dev Board #1 (Onboard Robot):** Central gait generator, sensor polling, UART servo master, ESP-NOW transmitter.
- **ESP32 Dev Board #2 (Base Station Bridge):** Dedicated ESP-NOW transceiver, USB-UART bridge to Raspberry Pi 5.
- **Raspberry Pi 5 (8GB RAM):** Base station supervisor, trajectory planning, telemetry visualization, web UI / ROS2 nodes.

Sensors:
- MPU6050 6-DoF IMU (GY-521 module, I^2C address `0x68`, posture/orientation tracking).
- HC-SR04P Ultrasonic Sensor (**3.3 V wide-voltage version**, safe for direct ESP32 GPIO, obstacle avoidance).

Cables & Fasteners:
- 10× ST-series 3-pin daisy-chain bus cables (+12 V / GND / DATA) + 3 spare cables.
- Silicone Dupont hookup jumpers, heat-shrink tubing, braided sleeving.
- Micro-USB / USB-C interface cables for programming and base station connectivity.

1.2 Return to Supplier (Dynamixel-Era Components)

The following components from the legacy design are deprecated and should be returned or repurposed:

- **Dynamixel XL330 Servos:** Inadequate torque for 10-link body load; high supply lead times in India; protocol incompatible with ST3215 bus.
- **ROBOTIS U2D2 Interface & Power Hub:** Dynamixel-exclusive TTL pinout and proprietary protocol converter.
- **Dynamixel Communication Cables:** Custom JST connectors incompatible with Waveshare ST-series sockets.

- **5 V Standalone Power Supply:** Incompatible voltage level; ST3215 motors require 12 V nominal rail.
- **12V→5V 5 A Buck Converter:** Bulky footprint; replaced with ultra-compact Mini560 (3 A).
- **XT60 to 5.5 mm Barrel Jack Cable:** Deprecated in favor of direct screw-terminal block connections.

2 Component Specifications

2.1 Waveshare ST3215 Smart Serial Servo

Parameter	Specification / Operating Limits
Operating Voltage Range	6.0 V – 12.6 V (Nominal: 11.1 V – 12.0 V)
Stall Torque	30 kg·cm ($\approx 2.94\,\mathrm{N \cdot m}$) @ 12 V
Stall Current	2.7 A per servo @ 12 V
Continuous Operating Current	≈ 1.0 A – 1.5 A under nominal snake locomotion
No-Load Speed	0.222 sec/60° (45 RPM) @ 12 V
Communication Protocol	Feetech SCServo Half-Duplex Asynchronous Serial TTL
Baud Rate	1,000,000 bps (1 Mbps) default
Addressing Capability	Up to 253 unique IDs on a single daisy-chain bus
Position Sensor	12-bit Contactless Magnetic Encoder (0.088° resolution, 360° multi-turn)
Real-Time Telemetry	Position, Velocity, Load/Current, Voltage, Internal Temperature

Table 1: Electrical and mechanical specifications of Waveshare ST3215 servo.

2.2 Bus Servo Adapter (A) — Waveshare SKU 25514

- **Power Input:** 9.0 V – 12.6 V via heavy-duty 2-pin screw terminal.
- **Logic Interface:** Microcontroller UART interface (TX, RX, 5V/3.3V, GND).
- **Automatic Flow Direction Control:** Onboard hardware direction-switching circuit enables transparent half-duplex single-wire TTL serial communication from full-duplex MCU UART without requiring manual GPIO direction pin manipulation.
- **Servo Outputs:** Dual 3-pin standard ST-bus ports wired in parallel for daisy-chain distribution.

2.3 Orange 3S LiPo 1500 mAh 30C Battery

- **Chemistry & Configuration:** Lithium Polymer (LiPo), 3S1P (3 series cells).
- **Nominal Voltage:** 11.1 V (3.7 V/cell).
- **Fully Charged Voltage:** 12.6 V (4.2 V/cell).
- **Low-Voltage Cutoff Limit:** 10.2 V total (3.4 V/cell) — hardware alarm trigger.
- **Capacity & Continuous Discharge:** 1500 mAh @ 30C = 45 A continuous per pack.
- **Peak Burst Discharge (10 s):** 50C = 75 A burst per pack.
- **Main Discharge Connector:** Genuine XT60 with high-temperature nylon housing.

2.4 Mini560-5V Step-Down Synchronous Buck Converter

- **Input Range:** 6.0 V – 32.0 V (operating on 11.1 V–12.6 V LiPo rail).
- **Output Voltage & Current:** Fixed 5.0 V ±1%, up to 3.0 A continuous (short bursts up to 4 A).
- **Efficiency:** ≈ 95% @ 12 V→5 V conversion at 1 A load.
- **Dimensions / Form Factor:** 18 mm × 12 mm × 4.5 mm, mass < 2 g.

3 Power Architecture

3.1 Dual Parallel LiPo Configuration

To satisfy peak dynamic current requirements across 10 active joints and extend operational run-time, two identical Orange 3S 1500 mAh 30C LiPo batteries are coupled in parallel through a balanced XT60 Y-harness.

Combined System Metric	Calculated Value / Capacity
Nominal Voltage	11.1 V (3 × 3.7 V)
Fully Charged Voltage	12.6 V (3 × 4.2 V)
Total Energy Capacity	3000 mAh (33.3 Wh)
Continuous Discharge Rating	30C × 2 = 60C Equivalent (180 A theoretical continuous)
Short-Term Peak Discharge	270 A peak (10 s burst)
Max Expected System Load	27 A (10× 2.7 A stall worst-case) ≪ LiPo capability

Table 2: Parallel battery configuration parameters.

3.2 Protection, Filtering & Safety Subsystems

1. **Individual Pack Fusing:** A 15 A mini-blade automotive fuse is installed on the positive lead of *each* individual LiPo pack *before* entering the parallel Y-junction. This prevents catastrophic reverse charging or thermal runaway in the event of an internal cell short in one pack.
2. **Master XT60 Loop Key:** A removable heavy-duty XT60 shorting plug acts as a physical master safety disconnect, placed immediately downstream of the combined parallel junction.
3. **LiPo Low-Voltage Buzzers:** Each pack maintains its own dedicated JST-XH balance port monitor with loud dual-piezo buzzers calibrated to sound at ≤ 3.4 V/cell.
4. **Transient Absorption Capacitor:** A 1000 µF / 25 V low-ESR electrolytic capacitor is mounted directly across the 12 V screw terminals of the Bus Servo Adapter (A) to buffer regenerative inductive kickback and dampen switching transients during rapid servo reversals.
5. **Logic Power Isolation:** The ESP32 and sensitive sensors receive clean 5.0 V via the Mini560 buck regulator, completely isolating MCU digital logic from motor voltage sag.

4 Wiring Details — On the Robot

4.1 12 V Main Power Distribution Bus

- **LiPo Pack #1 (+)** ⟶ 15 A Fuse #1 ⟶ Y-Harness Branch 1 (+)

- **LiPo Pack #2 (+)** $\longrightarrow$ 15 A Fuse #2 $\longrightarrow$ Y-Harness Branch 2 (+)
- **Combined Positive** $\longrightarrow$ XT60 Loop Key (Master Switch) $\longrightarrow$ Main +12 V Rail
- **Main +12 V Rail** branches directly into:
  1. Bus Servo Adapter (A) Screw Terminal `VIN` (+)
  2. Mini560-5V Buck Converter Input `IN+`
  3. 1000 µF 25 V Capacitor Anode (+)

4.2 System Ground (GND) Topology

All grounds are tied in a single-point star ground node at the battery negative junction:

- LiPo Pack #1 (−) and LiPo Pack #2 (−) $\longrightarrow$ XT60 Y-Harness Negative Node.
- Common GND connects to Bus Servo Adapter `GND` terminal, 1000 µF Capacitor Cathode (−), and Mini560 `IN-`.
- Mini560 `OUT-` connects to ESP32 `GND` pin, establishing unified signal and power reference.

4.3 UART & Servo Bus Routing

ESP32 Pin	Bus Servo Adapter Pin	Function
`GPIO17` (`UART2_TX`)	`RX`	Serial Commands from ESP32 to Bus
`GPIO16` (`UART2_RX`)	`TX`	Telemetry Feedback from Servos to ESP32
`5V`	`5V`	Logic Level Reference
`GND`	`GND`	Common Signal Ground

Table 3: ESP32 to Waveshare Bus Servo Adapter pin connections.

Servo Daisy-Chain Path:

Adapter 3-Pin Port $\xrightarrow{\text{Cable 1}}$ Servo #1 $\xrightarrow{\text{Cable 2}}$ Servo #2 $\longrightarrow \cdots \longrightarrow$ Servo #9 $\xrightarrow{\text{Cable 10}}$ Servo #10

The output port of Servo #10 is left open (or protected with an insulated dummy dust plug).

5 Wiring Details — Base Station

6 First Power-Up Sequence

Mandatory Safety Warning

Execute Steps 1–3 **strictly** prior to attaching any servo or logic processor. Do not insert the XT60 loop key until all resistance checks confirm no ground-to-power shorts exist.

Step 1: Pre-Assembly Visual Inspection (15 min)
- Confirm 15 A blade fuses are seated in both positive harness branches.
- Verify the XT60 Loop Key is removed (system in open circuit).
- Inspect LiPo packs for swelling or mechanical punctures; verify cell voltages on buzzer alarms ($\geq$ 3.8 V/cell).

Base Station Architecture

- **Raspberry Pi 5 (8GB):** Powered via official 27 W (5.1 V, 5 A) USB-C power supply. Runs Linux (Ubuntu 24.04 / Raspberry Pi OS), ROS 2 communication nodes, Web GUI dashboard, and inverse kinematics solvers.
- **ESP32 Dev Board #2 (Radio Gateway):** Connected directly to a USB 3.0 port on the Raspberry Pi 5. Powered via 5 V USB VBUS; enumerates as `/dev/ttyUSB0` or `/dev/ttyACM0` @ 115200 or 921600 baud.
- **Communication Pipeline:**

$$\text{ESP32 \#1 (Robot)} \underset{<10\,\text{ms}}{\overset{\text{ESP-NOW (2.4 GHz)}}{\rightleftharpoons}} \text{ESP32 \#2 (Bridge)} \underset{1\,\text{Mbps}}{\overset{\text{USB Serial}}{\rightleftharpoons}} \text{Raspberry Pi 5}$$

- Check capacitor polarity: stripe / negative lead to `GND`, positive lead to `+12V`.

Step 2: Megohm Continuity & Short-Circuit Test (10 min) • Disconnect ESP32 from the 5 V buck regulator.

- Set digital multimeter to Resistance / Megohm range (MΩ).
- Measure resistance across the main 12 V rail and GND rail. Resistance must register $> 10\,\text{M}\Omega$ (charging capacitor may show transient low reading before settling at high MΩ).
- If reading is $< 1\,\text{M}\Omega$, halt immediately and inspect wiring harnesses for solder bridges.

Step 3: Initial Power-ON with Voltage Verification (20 min) • Insert the XT60 Loop Key. No spark, popping, or smoke should occur.

- Measure voltage at the Bus Servo Adapter screw terminal: must read 11.1 V – 12.6 V.
- Measure voltage across Mini560 output terminals: must read $5.0\,\text{V} \pm 0.1\,\text{V}$.
- Disconnect Loop Key before proceeding.

Step 4: Single-Servo Daisy-Chain Staging (15 min) • Connect only Servo #1 to the adapter bus output port. Insert Loop Key.

- Confirm the servo LED indicators blink or steady-state illuminate. Verify the servo body remains at ambient temperature.
- Incrementally attach Servos #2 through #10 one by one, checking thermal state at each step.

Step 5: Load ESP32 Test Firmware (10 min) • Flash the servo ID read-back firmware to ESP32 #1 via USB.

- Verify bidirectional half-duplex UART ping responses across the 1 Mbps serial bus.

Step 6: ESP-NOW Wireless Pairing (10 min) • Flash the receiver firmware to ESP32 #2 (connected to base station) and monitor serial output at 115200 baud.

- Power up ESP32 #1 on the robot; verify real-time packet stream reception with $< 15\,\text{ms}$ interval.

7 Servo Integration & ID Assignment

All Waveshare ST3215 servos are pre-programmed from the factory with a default ID of **1**. To prevent bus collisions, each servo must be individually programmed with a unique identifier

(ID $\in [1, 10]$) prior to final mechanical assembly.

7.1 Sequential ID Assignment Procedure

1. Disconnect all servos from the bus adapter.
2. Connect **only one** target servo to the adapter port.
3. Flash and run the ID Assignment sketch on ESP32 #1.
4. Input the desired target ID (1–10) via the Serial Monitor.
5. The sketch writes the new ID into the servo's non-volatile EEPROM register (Register `0x05`) and executes an EEPROM lock.
6. Unplug the programmed servo, label it physically (e.g., Joint 1 to Joint 10), and repeat for the remaining units.

7.2 ID Assignment Source Code

```
#include <Arduino.h>
#include <HardwareSerial.h>

// ESP32 UART2 (GPIO16=RX, GPIO17=TX)
HardwareSerial ServoSerial(2);

void setServoID(uint8_t currentID, uint8_t newID) {
    // Feetech SCServo Protocol Packet:
    // [0xFF, 0xFF, ID, LENGTH, INSTRUCTION, PARAM_REG, VALUE, CHECKSUM]
    uint8_t cmd[7];
    cmd[0] = 0xFF;              // Header 1
    cmd[1] = 0xFF;              // Header 2
    cmd[2] = currentID;         // Current ID (Default = 1)
    cmd[3] = 0x04;              // Packet Length (Cmd + Reg + Value + Checksum)
    cmd[4] = 0x03;              // Instruction: WRITE_DATA (0x03)
    cmd[5] = 0x05;              // Register Address: ID register (0x05)
    cmd[6] = newID;             // Value: New Target ID

    // Compute Checksum: ~(ID + LENGTH + INSTRUCTION + PARAM_REG + VALUE) & 0xFF
    uint8_t sum = cmd[2] + cmd[3] + cmd[4] + cmd[5] + cmd[6];
    uint8_t checksum = ~sum;

    uint8_t packet[8];
    memcpy(packet, cmd, 7);
    packet[7] = checksum;

    // Transmit packet at 1 Mbps
    ServoSerial.write(packet, 8);
    ServoSerial.flush();

    Serial.printf("Transmitted ID change request: %d -> %d (Checksum: 0x%02X)\n",
                  currentID, newID, checksum);
}

void setup() {
    Serial.begin(115200);
    ServoSerial.begin(1000000, SERIAL_8N1, 16, 17); // 1 Mbps, RX=16, TX=17
    delay(1000);
    Serial.println("\n=== ST3215 Servo ID Programming Tool ===");
    Serial.println("Enter target ID (1 - 10) in the Serial Monitor:");
}

```

```
void loop() {
    if (Serial.available() > 0) {
        int inputVal = Serial.parseInt();
        if (inputVal >= 1 && inputVal <= 10) {
            Serial.printf("Assigning target ID: %d ...\n", inputVal);
            setServoID(1, (uint8_t)inputVal);
            delay(500);
            Serial.println("Programming complete. Power cycle servo to verify.");
        } else if (inputVal != 0) {
            Serial.println("Error: Valid ID range is 1 to 10.");
        }
    }
}
```

Listing 1: ESP32 Arduino Sketch for ST3215 Servo ID Assignment.

8 Sensor Integration

8.1 MPU6050 6-Axis IMU (Orientation Feedback)

Mounted in the head/lead segment of the snake robot to estimate pitch, roll, and lateral acceleration for terrain-adaptive gait modulation.

- **Module:** GY-521 Breakout Board (with onboard 3.3 V LDO regulator).
- **Interface:** I^2C Standard/Fast Mode (400 kHz), default 7-bit address `0x68` (AD0 pin pulled low).
- **Pin Mapping:** `VCC` → ESP32 3.3 V, `GND` → ESP32 GND, `SCL` → `GPIO22`, `SDA` → `GPIO21`.

8.2 HC-SR04P Ultrasonic Sensor (Obstacle Detection)

Hardware Compatibility Requirement

Ensure the **HC-SR04P** (wide voltage 3.0 V–5.5 V) module is utilized. Standard HC-SR04 modules output 5 V logic on the `ECHO` pin, which will cause permanent over-voltage damage to the ESP32 input stage (3.3 V max tolerance).

- **Pin Mapping:** `VCC` → ESP32 3.3 V (or 5 V), `GND` → ESP32 GND, `TRIG` → `GPIO26` (Output), `ECHO` → `GPIO27` (Input).

8.3 Unified Sensor Polling Code

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

Adafruit_MPU6050 mpu;
const int TRIG_PIN = 26;
const int ECHO_PIN = 27;

void setup() {
    Serial.begin(115200);
    Wire.begin(21, 22); // SDA=21, SCL=22

    if (!mpu.begin()) {
        Serial.println("Failed to find MPU6050 chip!");
        while (1) { delay(10); }
```

```
    }
    mpu.setAccelerometerRange(MPU6050_RANGE_4_G);
    mpu.setGyroRange(MPU6050_RANGE_500_DEG);
    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    Serial.println("Sensors initialized successfully.");
}

void loop() {
    // 1. Read IMU
    sensors_event_t a, g, temp;
    mpu.getEvent(&a, &g, &temp);

    // 2. Read Ultrasonic Distance
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000); // 30ms timeout
    float distance_cm = (duration == 0) ? -1.0 : (duration / 2.0) / 29.1;

    // Output Telemetry
    Serial.printf("IMU Accel: [%.2f, %.2f, %.2f] m/s^2 | Dist: %.1f cm\n",
                  a.acceleration.x, a.acceleration.y, a.acceleration.z, distance_cm);
    delay(50);
}
```

Listing 2: Sensor Integration Sketch for ESP32 Onboard.

9 ESP-NOW Wireless Telemetry & Command Link

ESP-NOW is a connectionless, peer-to-peer MAC-layer protocol developed by Espressif, operating on the 2.4 GHz ISM band.

- **Packet Payload:** Up to 250 bytes per frame.
- **Latency:** < 10 ms deterministic round-trip time.
- **Throughput:** 50 Hz bidirectional telemetry and gait parameter updates.

9.1 Phase 1: MAC Address Discovery

Run the following helper code on both boards to determine their factory Wi-Fi MAC addresses:

```
#include <WiFi.h>
#include <esp_wifi.h>

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_MODE_STA);
    delay(100);
    uint8_t mac[6];
    esp_wifi_get_mac(WIFI_IF_STA, mac);
    Serial.printf("Device Station MAC: %02X:%02X:%02X:%02X:%02X:%02X\n",
                  mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
}

```

```
void loop() {}
```

Listing 3: ESP32 MAC Address Extraction Script.

9.2 Phase 2: Robot Transmitter Firmware (ESP32 #1)

```
#include <esp_now.h>
#include <WiFi.h>

// Replace with actual Base Station ESP32 #2 MAC address
uint8_t baseStationMAC[] = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF};

typedef struct __attribute__((packed)) {
    float roll;
    float pitch;
    float yaw;
    float head_distance_cm;
    uint16_t servo_current_mA[10];
    uint16_t servo_positions[10];
    uint8_t battery_voltage_deciV; // e.g. 118 = 11.8V
} RobotTelemetryPacket;

RobotTelemetryPacket txData;

void onDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
    // Optional delivery acknowledgment logging
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_MODE_STA);

    if (esp_now_init() != ESP_OK) {
        Serial.println("Error initializing ESP-NOW");
        return;
    }

    esp_now_register_send_cb(onDataSent);

    esp_now_peer_info_t peerInfo = {};
    memcpy(peerInfo.peer_addr, baseStationMAC, 6);
    peerInfo.channel = 1;
    peerInfo.encrypt = false;

    if (esp_now_add_peer(&peerInfo) != ESP_OK) {
        Serial.println("Failed to add Base Station peer");
    }
}

void loop() {
    // Populate sample telemetry from sensors and ST3215 feedback
    txData.roll = 3.25;
    txData.pitch = -1.10;
    txData.head_distance_cm = 42.5;
    txData.battery_voltage_deciV = 121; // 12.1V

    esp_err_t result = esp_now_send(baseStationMAC, (uint8_t *)&txData, sizeof(txData));
    delay(20); // 50 Hz transmission cycle
}
```

Listing 4: Transmitter Firmware running on Onboard Robot ESP32.

9.3 Phase 3: Base Station Receiver Firmware (ESP32 #2)

```
#include <esp_now.h>
#include <WiFi.h>

typedef struct __attribute__((packed)) {
    float roll;
    float pitch;
    float yaw;
    float head_distance_cm;
    uint16_t servo_current_mA[10];
    uint16_t servo_positions[10];
    uint8_t battery_voltage_deciV;
} RobotTelemetryPacket;

RobotTelemetryPacket rxData;

void onDataRecv(const esp_now_recv_info_t *info, const uint8_t *incomingData, int len) {
    if (len == sizeof(RobotTelemetryPacket)) {
        memcpy(&rxData, incomingData, sizeof(RobotTelemetryPacket));

        // Forward structured telemetry over high-speed USB-UART to Raspberry Pi 5
        Serial.printf("TELEMETRY,%.2f,%.2f,%.1f,%d\n",
                      rxData.roll, rxData.pitch, rxData.head_distance_cm,
                      rxData.battery_voltage_deciV);
    }
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_MODE_STA);

    if (esp_now_init() != ESP_OK) {
        Serial.println("ESP-NOW Init Failed");
        return;
    }

    esp_now_register_recv_cb(onDataRecv);
    Serial.println("ESP-NOW Base Station Receiver Online. Awaiting packets...");
}

void loop() {
    // Receiver loop operates asynchronously via hardware interrupt callbacks
    delay(10);
}
```

Listing 5: Receiver Bridge Firmware on Base Station ESP32.

10 Project Status & Next Milestones

Document Summary & Status

- **Status:** Phase 2 (Electrical Redesign, Component Selection, Safety Architecture) is **Complete**.
- **Readiness:** Phase 3 (Harness Fabrication, 3D Print Integration, Subsystem Bench Testing) is ready to commence.
- **Next Review Milestone:** After physical arrival of Waveshare ST3215 motors, adapter board, and completion of the first 2-segment mechanical articulated joint test bench.